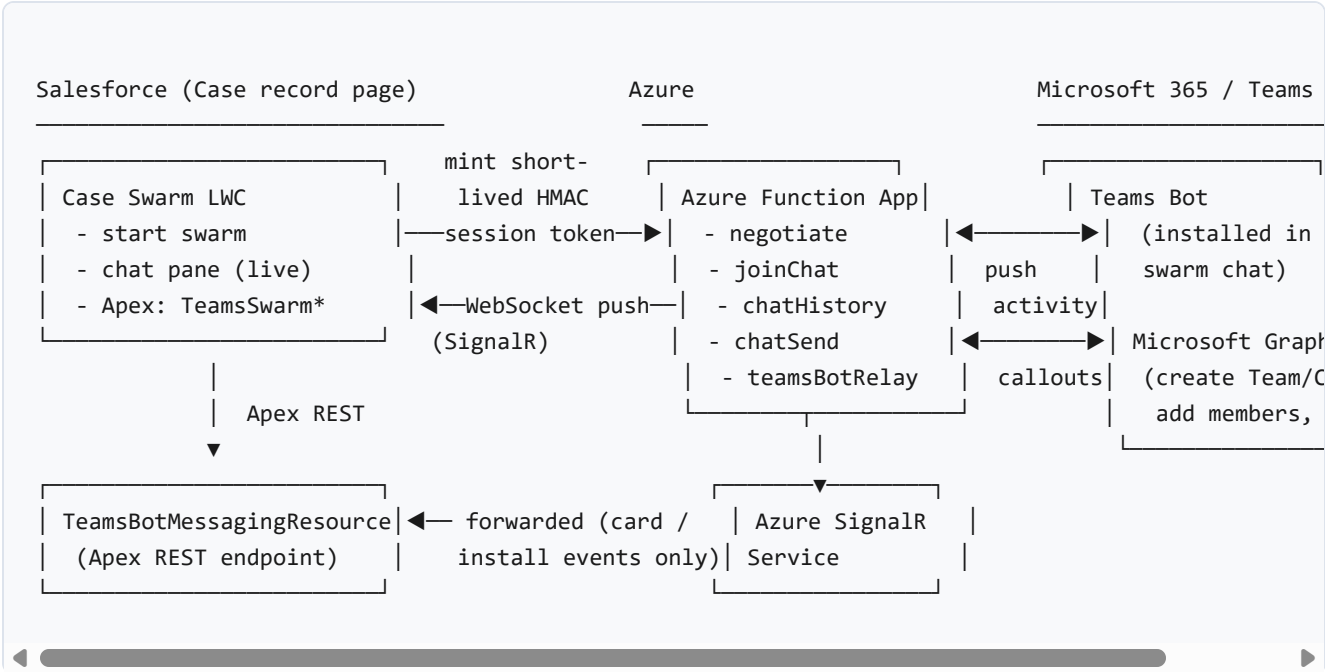


Case Swarm — Salesforce ↔ Microsoft Teams Real-Time Chat

Solution overview for business and technical architecture review — how it works, what's required on each side, cost, platform-limit impact, known limitations, and the implementation challenges encountered building it.

What this delivers: an agent working a Salesforce Case can pull in subject-matter experts via a Microsoft Teams conversation ("swarm"), edit the Case directly from an interactive card inside that Teams chat, and hold a live, two-way chat with Teams participants without ever leaving Salesforce — messages appear instantly in both directions, not on a delay.

1. How it works — architecture overview



Two Microsoft auth surfaces are used for different purposes: **Microsoft Graph** (create/manage the Team or Chat, add members, install the bot) and the **Bot Framework Connector** (post the interactive card, send/receive live chat messages) — Graph's application-only token cannot post live chat messages, so the Bot Framework is used specifically for that.

The three ways this can run (feature-flagged, no code branching required to switch)

Mode	Read (Teams → Salesforce)	Send (Salesforce → Teams)	When used
------	---------------------------	---------------------------	-----------

Apex polling (original)	Apex calls Graph every 5s	Apex → Bot Framework Connector	Fallback / <code>Use_Azure_Chat_Bridge__c = false</code>
Azure bridge polling	LWC polls Azure <code>chatHistory</code> every 5s (Azure calls Graph)	LWC → Azure <code>chatSend</code> → Bot Framework	<code>Use_Azure_Chat_Bridge__c = true</code> , <code>Use_Azure_SignalR__c = false</code>
SignalR push (current)	Instant push via Azure SignalR; one-time history load only, no recurring poll	LWC → Azure <code>chatSend</code> → Bot Framework	<code>Use_Azure_SignalR__c = true</code> — live in production

All three modes share the same Case, custom objects, and Apex controller — switching between them is a custom metadata toggle, not a redeploy of business logic.

2. What's required — Salesforce side

Component	Purpose
Custom objects: <code>Swarm_Message__c</code> , <code>Swarm_Usage_Log__c</code>	Chat transcript backfill and per-session usage tracking
Custom metadata: <code>Teams_Bot_Config__mdt</code>	Central config — bot app id, Teams app catalog id, Azure base URL, shared HMAC secret, and the two feature-flag checkboxes above
Apex: <code>TeamsSwarmController</code> , <code>TeamsSwarmChatController</code> , <code>TeamsSwarmService</code> , <code>TeamsBotMessagingResource</code> , <code>TeamsBotJwtValidator</code> , <code>TeamsBotConversationService</code> , supporting Queueables	Swarm creation, chat bridge session minting, Graph/Bot Framework service calls, inbound bot webhook handling, hand-rolled JWT validation (Apex has no native JWT library)
LWCs: <code>caseSwarm</code> , <code>caseSwarmChatCore</code> , <code>caseSwarmChatWidget</code> , <code>caseSwarmChatUtility</code> , <code>caseSwarmChat</code> , <code>swarmUsageTracker</code>	Swarm creation UI, the chat pane itself (embedded in 3 different placements), shared usage-tracking module
Static Resources: <code>signalrLwc.js</code> (+ raw vendor source)	A specially patched build of Microsoft's <code>@microsoft/signalr</code> browser client, hardened for Salesforce's Lightning Web Security sandbox (see Challenges)
Named/External Credentials: <code>MS_Graph</code> , <code>MS_Bot_Framework</code>	Outbound OAuth 2.0 client-credentials auth from Apex to Microsoft Graph and the Bot Framework Connector
CSP Trusted Sites (Connect-src)	Allow the LWC to call the Azure Function App and open a WebSocket to Azure SignalR Service directly from the browser
Permission Set: <code>Teams_Swarm_User</code>	Field/object access for agents using swarm + chat features

Connected App (Client Credentials Flow)	Lets the Azure relay authenticate to Salesforce as a real internal user (not a Guest User) to call the inbound bot webhook endpoint
---	---

3. What's required — Azure side

Resource	Purpose
Azure Function App (Consumption plan, Node.js v4 model)	Hosts 5 functions: <code>teamsBotRelay</code> (inbound webhook fix-up), <code>negotiate</code> + <code>joinChat</code> (SignalR connection bootstrap), <code>chatHistory</code> (one-time history load), <code>chatSend</code> (outbound message post)
Azure SignalR Service	Managed WebSocket push service — delivers new Teams messages to the browser instantly instead of polling
Storage Account	Required by Azure Functions for its own bookkeeping (not used for chat data — nothing is persisted; Teams remains the sole source of truth)
Application Insights	Logging/monitoring for the Function App
App settings (secrets)	<code>SALESFORCE_*</code> (Connected App creds), <code>MICROSOFT_APP_*</code> (bot creds), <code>GRAPH_*</code> (Graph app creds), <code>CHAT_BRIDGE_HMAC_SECRET</code> (shared with Salesforce), <code>AzureSignalRConnectionString</code>

4. What's required — Microsoft 365 / Teams side

Item	Purpose
Azure AD App Registration	Identity used for both Graph calls and the bot; must be single-tenant-aware (see Challenges) with Graph application permissions granted and admin-consented
Azure Bot resource	Registers the bot with the Bot Framework; messaging endpoint points at the Azure relay, not Salesforce directly
Teams app manifest + package, uploaded to the tenant app catalog	Declares the bot's chat scope and the Resource-Specific Consent (RSC) permission <code>ChatMessage.Read.Chat</code> — required for the bot to see chat messages that don't @mention it
Teams Admin Center — Permission policies	Must explicitly allow this custom/LOB app for the relevant users; blocked by default in some tenants (see Challenges)
Tenant admin / RSC consent	Application-type RSC permissions require at least one interactive install (via the Teams client, by an authorized user) to bootstrap consent before automated Graph-based installs succeed for other chats

5. Cost

Item	Estimated cost	Notes
Azure Function App (Consumption plan)	~\$0	Free monthly grant (~1M executions) comfortably covers this workload at realistic usage
Azure SignalR Service (Free tier)	\$0	20 concurrent connections / 20,000 messages per day — sufficient unless usage grows past ~20 simultaneously open chats org-wide
Storage Account	Negligible (cents/month)	Only used for Function App bookkeeping, no chat data stored
Application Insights	~\$0	Free tier sufficient at this scale
Salesforce	\$0 additional licensing	Built entirely on standard platform features already licensed (Apex, LWC, Custom Objects/Metadata)
Total estimated recurring cost	Effectively \$0/month at current scale	Revisit if usage grows to many dozens of concurrent swarm chats (Azure SignalR Standard tier: roughly \$1–2/unit/day beyond the free tier)

6. Platform limits — impact on each side

Limit	Impact in this design
Salesforce Daily API Request allocation (org-wide, 24h)	Not consumed for messaging — the relay acks plain Teams chat messages without forwarding them to Salesforce; only rare events (bot install, card edits) are forwarded
Salesforce Apex callouts (100/transaction)	Not used for live chat in SignalR mode — session tokens are minted locally in Apex with no external call; only swarm creation and message sending make callouts, each a single user-triggered action
Microsoft Graph chat-message-list throttle (10 requests/10 seconds per app/tenant)	Eliminated in SignalR mode — this limit was hit live under the earlier polling design (confirmed via a live "API calls quota exceeded" error) and was the original motivation to move to push-based delivery
Azure SignalR Service (Free tier: 20 connections / 20,000 messages/day)	The one limit to monitor going forward — bound by how many chat widgets are open simultaneously across the org, not by message volume
Azure Function executions	Comfortably within free grant at any realistic usage level for this feature

7. Known limitations

- **Team/Channel swarms don't get the interactive edit-in-Teams card** — only Chat (group chat) swarms do; posting into a channel uses a different Graph/Bot Framework flow that isn't built.
- **No automated cleanup** of the underlying Microsoft 365 Team/Group/Chat when a Case's swarm is cleared or the Case is closed — these accumulate in the tenant unless manually managed.
- **Teams group-chat bots only receive @mentioned messages by default** — receiving every message (not just mentions) required declaring and successfully consenting the `ChatMessage.Read.Chat` Resource-Specific Consent permission, which is Application-type and needs the bootstrap/consent process described in Challenges below.
- **No persisted chat transcript during the live session** — Teams is treated as the sole source of truth while chatting; a transcript is only backfilled into Salesforce (`Swarm_Message__c`) at end-of-session via a one-time pull.
- **Usage tracking is approximate, not authoritative** — session counters are tracked client-side in the browser and reset if the tab closes before being read; they're for understanding usage patterns, not billing-grade audit.
- **Catalog id churn** — if the Teams app package is ever deleted and re-uploaded (rather than updated in place), it gets a new catalog id, which breaks bot installs into new chats until `Teams_Bot_Config__mdt` is updated, and re-installation is required in every existing swarm chat.

8. Implementation challenges encountered

Included for technical review — this reflects real issues hit and resolved during build-out, useful context for estimating effort on similar integrations.

Salesforce blocks direct inbound webhooks from Bot Framework. Salesforce's REST dispatcher intercepts any standard `Authorization` header on an Apex REST endpoint and tries to validate it as a Salesforce session token — a fixed platform behavior, not configurable. Bot Framework always sends its JWT in that exact header. Resolved with a minimal Azure Function relay that moves the JWT to a different header before forwarding.

Microsoft Graph cannot send live chat messages under application-only auth (returns 401) — sending had to go through the Bot Framework Connector API instead, a separate credential/auth flow from Graph entirely.

Microsoft Graph's chat-message throttle (10 requests/10 seconds) was hit live under an earlier polling-based design, which directly motivated the move to SignalR push.

Three separate Lightning Web Security (LWS) sandbox incompatibilities with the vendored SignalR client library, each requiring a distinct fix:

- The bare `self` global is unreliable inside LWS — found first in the integration's own code, then again inside the vendored bundle's own internal UMD wrapper, requiring the entire bundle to be patched to shadow `self` locally rather than touch LWS's real binding.
- Salesforce's platform Content Security Policy hard-blocks `eval()` / `new Function()` with no way for an org admin to relax it — a dead-code fallback path inside webpack's bundled "get global object" helper was silently triggering this block inside the sandbox and had to be removed entirely.
- The chat component is mounted in three different places on a page simultaneously (a card, a floating widget, and a Utility Bar item); each independently loading the same script concurrently caused unreliable load failures until the load was made a single shared, page-wide operation.

Teams Resource-Specific Consent (RSC) for Application permissions required an interactive bootstrap. Declaring `ChatMessage.Read.Chat` in the manifest wasn't sufficient on its own — the fully automated, headless Graph-based bot install failed with `ResourceSpecificPermissionsMismatch` until the app was also installed at least once via the interactive Teams client, which is the only path that can show the RSC consent prompt.

Tenant-level Teams Admin Center permission policy blocked the custom app outright, independent of any of the above — required Teams/Global admin intervention to explicitly allow the app before any install (automated or manual) could succeed.

9. Summary for business

The end-to-end solution is live and working: agents can start a Teams conversation from a Case, bring in experts, edit the Case inline from Teams, and chat live in both directions — all with effectively **zero incremental recurring cost** at current usage levels, and without pressuring any Salesforce or Microsoft platform limit under normal operation. The main ongoing considerations are operational rather than financial: monitoring Azure SignalR's free-tier connection cap if usage grows significantly, and periodically cleaning up Teams resources for closed cases, since that isn't automated today.

Document reflects the solution as implemented and verified end-to-end at the time of writing.